Technical Specification: ML-Based Network Intrusion Detection System (NIDS)

Build guide for an autonomous coding agent. Version 1.0. Scope: this project only (offline, dataset-driven NIDS with FastAPI backend and Streamlit SOC dashboard).

0. How the Agent Must Use This Document

0.1 Normative words. MUST / MUST NOT = mandatory. SHOULD = default unless there is a documented reason (record it in `docs/DECISIONS.md`). MAY = optional.

0.2 Precedence. This specification resolves ambiguities in the original project proposal. Where this document is silent, choose the simplest option that satisfies the proposal and record the choice in `docs/DECISIONS.md` (one line: decision, reason, date).

0.3 Non-negotiable rules.

ID	Rule
R1	NEVER fabricate dataset columns, row counts, class counts, metrics, alerts, or traffic. Every number shown in the README, model card, or dashboard MUST be read from a generated artifact file (JSON/Parquet) produced by the pipeline. Before results exist, show the literal text `Not yet generated. Run the pipeline.`
R2	Discover, then assert. Inspect the real dataset first, write `reports/schema_report.json`, and make code assert the expectations in this spec. If reality differs, STOP and report (see 0.4). Do not silently adapt.
R3	Fit anything that learns from data (imputer medians, scaler, feature selection, thresholds' training part, model) on the TRAIN split only.

ID	Rule
R4	The test split is used exactly once, for final reporting, after all model selection and threshold choice are frozen on the validation split. The final evaluation script MUST refuse to run if `models/FROZEN` marker file is absent, and MUST write `reports/test_metrics.json` only once (refuse to overwrite unless `--force` is passed; log the override).
R5	Training and inference MUST use the same preprocessing code, saved inside one serialized pipeline object. No duplicated preprocessing logic in the API or dashboard.
R6	Offline analysis only. NO packet capture, NO active scanning, NO live monitoring, NO network access to third-party hosts at runtime.
R7	Any demo or synthetic data MUST be labelled in the UI and in file metadata as `DEMO` or `SYNTHETIC`. Synthetic data is allowed only for unit tests and UI smoke tests, never for reported model performance.
R8	The model output is a **model score**, not a calibrated probability of attack, unless calibration has been evaluated (Section 8.6). A prediction is never called a confirmed attack.
R9	Work phase by phase (Section 17). Run each phase's acceptance checks. Do not start the next phase while checks fail.
R10	Keep it simple. No Docker requirement, no message queues, no task runners, no extra services. PostgreSQL is optional and off by default.

0.4 When to STOP and ask the human. Output a clear message (what was expected, what was found, what is needed) and halt in these cases: (a) no CSV files in `data/raw/cicids2017/`; (b) a Label value is not in `config/label_map.yaml`; (c) fewer than 90% of expected feature columns found; (d) timestamps cannot be parsed for more

than 1% of rows AND the row-order fallback (5.5) is not applicable; (e) available RAM is too low to load a single day file; (f) any acceptance check in Section 17 fails after two fix attempts.

0.5 Progress log. After each phase append to `docs/PROGRESS.md`: phase number, date, commands run, acceptance results (pass/fail), known issues.

1. Product Definition

1.1 Purpose. Analyse recorded network-flow records, classify each flow as benign or malicious (binary), then as an attack family (multiclass), and present results in a dark SOC-style dashboard.

1.2 In scope (v1). CSV dataset ingestion and validation; leakage-aware splitting; training and comparison of Logistic Regression, Random Forest, XGBoost, Isolation Forest; evaluation reports; saved inference bundle; FastAPI service (`/health` , `/model/info` , `/predict` , `/predict/batch` , `/explain`); Streamlit dashboard with six pages; CSV upload and analysis; downloadable predictions; SHAP explanations; file-based run storage; optional PostgreSQL storage; tests; documentation.

1.3 Out of scope (v1). Live capture, PCAP parsing, CICFlowMeter execution, real-time streaming, user accounts/authentication UI, cloud deployment, retraining from the dashboard, UNSW-NB15 support (an adapter interface is defined, implementation deferred).

1.4 Glossary.

Term	Meaning
Flow	One row: a bidirectional summary of packets between two endpoints, as produced by the dataset authors' flow tool.
Malicious (predicted)	Model score at or above the operating threshold. NOT a confirmed attack.
Attack family	Grouped label defined in `config/label_map.yaml` .
Model score	Output of the model's positive-class scoring (for example forest vote fraction). Uncalibrated unless Section 8.6 says otherwise.

Term	Meaning
FPR	False positive rate = FP / (FP + TN) on benign flows.
Operating threshold	Score cut-off chosen on the validation split (Section 8.5).

2. Environment and Dependencies

2.1 Platform. Python 3.11 (3.10 to 3.12 supported). Windows, macOS, or Linux. Minimum 8 GB RAM (16 GB recommended), 10 GB free disk. No GPU. No paid services.

2.2 Setup commands.

Linux/macOS:

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements-dev.txt
pip install -e .
```

Windows (PowerShell):

```
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements-dev.txt
pip install -e .
```

2.3 Runtime dependencies (`requirements.txt`). Version ranges are required; after the first fully passing run, the agent MUST write the exact resolved versions to `requirements.lock` using `pip freeze`.

Package	Range	Purpose
numpy	>=1.26,<3	arrays
pandas	>=2.2,<3	tables
pyarrow	>=15	Parquet I/O
scikit-learn	>=1.4,<2	models, pipeline, metrics

Package	Range	Purpose
xgboost	>=2.0,<4	gradient boosting
joblib	>=1.3	model serialization
pyyaml	>=6	config
pydantic	>=2.6,<3	schemas
pydantic-settings	>=2.2	environment config
fastapi	>=0.110	API
uvicorn[standard]	>=0.29	ASGI server
python-multipart	>=0.0.9	form parsing (kept for future file upload)
streamlit	>=1.36	dashboard (needs `st.navigation`)
plotly	>=5.20,<7	charts
requests	>=2.31	dashboard to API calls
shap	>=0.45	explanations
matplotlib	>=3.8	static report figures
networkx	>=3.2	optional network graph

2.4 Dev dependencies (`requirements-dev.txt`, includes `-r requirements.txt`). pytest>=8, pytest-cov, httpx (FastAPI TestClient), ruff, black.

2.5 Optional (`requirements-postgres.txt`). sqlalchemy>=2.0, psycopg[binary]>=3.1.

2.6 Packaging. `pyproject.toml` defines the package `nids` located in `src/nids/` (src layout) so that `import nids` works from backend, frontend, scripts, and tests without path hacks. Forbidden: `sys.path.append` hacks.

3. Dataset Specification

3.1 Decision. Primary dataset: **CIC-IDS2017** (Canadian Institute for Cybersecurity). Reasons: it contains the attack categories named in the proposal (brute force, DoS, DDoS, port scan, web attacks, botnet); the labelled-flows distribution includes IP addresses and timestamps needed for the Alerts, Investigation and Network Activity pages; it is widely used so results can be compared with literature. UNSW-NB15 is not implemented in v1. A

`DatasetAdapter` abstract class in `src/nids/data/adapters.py` defines the extension point (methods: `list_files()`, `read_file()`, `canonical_columns()`, `label_map()`), with `CicIds2017Adapter` as the only implementation.

3.2 Two distributions exist. The loader MUST detect which one it has.

Variant	Typical folder name	Contains	Support level
A	GeneratedLabelledFlows / TrafficLabelling	metadata columns (Flow ID, Source IP, Source Port, Destination IP, Destination Port, Protocol, Timestamp) + flow features + Label	REQUIRED, primary
B	MachineLearningCVE	flow features + Label only (no IPs, no timestamps)	SHOULD support in degraded mode: IP/time-dependent dashboard panels are disabled with an explanatory message, and splitting uses row-order blocks (5.5)

Detection rule: variant is A if both a source-IP-like and a timestamp-like column exist after name normalization (aliases in `config/columns.yaml`); otherwise B. Mixed folders (some files A, some B) MUST raise an error.

3.3 Acquisition (human step). The dataset is downloaded manually from the official CIC (University of New Brunswick) page. The agent MUST NOT attempt to download it. The agent MUST create `data/raw/cicids2017/README.txt` instructing the user to place the CSV files there, and MUST exit with the 0.4(a) message if the folder has no `.csv` files. Raw data and processed data MUST be git-ignored.

3.4 Characteristics the agent must VERIFY (not assume). Each item below is a commonly reported property of this dataset. The validation step (5.2) must detect each, record the observed result in `reports/schema_report.json`, and the cleaning step (5.3) must handle it.

1. Column names may contain leading/trailing spaces (for example `Label`).

2. A column name may appear twice in variant A (reported: `Fwd Header Length`).
3. `Flow Bytes/s` and `Flow Packets/s` may contain `Infinity`, `-Infinity` or NaN.
4. Some columns that should be non-negative contain negative values (reported for flow duration, IAT and initial-window columns). Do NOT delete these rows; record counts only.
5. Some label strings contain a non-ASCII character in place of a dash (web attack labels).
6. Files may contain blank rows, rows with missing Label, or a repeated header row.
7. Many exact duplicate rows exist.
8. Timestamp formatting may be inconsistent and may lack AM/PM so that afternoon hours are ambiguous (see 5.5).
9. Severe class imbalance; some attack classes have only tens of rows; several attack classes occur only on one day.
10. The dataset is a lab-generated 2017 capture; findings do not prove performance on modern production traffic (state this in README and dashboard footer).

3.5 Label mapping (`config/label_map.yaml`). Two levels: `fine` (normalized label string) and `family`. Normalization of a raw label: strip whitespace; replace every run of non-ASCII characters (with surrounding spaces) by `-`; collapse repeated spaces; compare case-insensitively.

Family (multiclass target)	Fine labels mapped to it
BENIGN	BENIGN
DoS	DoS Hulk, DoS GoldenEye, DoS slowloris, DoS Slowhttptest
DDoS	DDoS
PortScan	PortScan
BruteForce	FTP-Patator, SSH-Patator
WebAttack	Web Attack - Brute Force, Web Attack - XSS, Web Attack - Sql Injection
Bot	Bot
RareOther	Infiltration, Heartbleed

A raw label that normalizes to a string absent from the file MUST raise `UnmappedLabelError` listing all offenders (it MUST NOT default to malicious or benign). Binary target: `BENIGN` to 0, every other family to 1. The `fine` level is retained in the data for per-attack reporting (8.4) even when training on families.

3.6 Canonical column names. After loading, every column name is normalized: strip, collapse internal whitespace runs to one space, keep original case and punctuation (so `Flow Bytes/s` stays). Metadata aliases (for example `Src IP`, `Dst IP`, `Src Port`, `Dst Port`) are mapped to the canonical metadata names through `config/columns.yaml`. Canonical metadata names: `Flow ID`, `Source IP`, `Source Port`, `Destination IP`, `Destination Port`, `Protocol`, `Timestamp`.

3.7 Feature eligibility. Columns that are NEVER model features: `Flow ID`, `Source IP`, `Destination IP`, `Source Port`, `Timestamp`, `Label`, and any derived columns (`label_fine`, `label_family`, `label_binary`, `source_file`, `row_id`, `split`). Reason: identifiers and time leak the capture setup, not behaviour. `Destination Port` and `Protocol` ARE eligible features by default (service identity is legitimate signal) but carry shortcut risk; the agent MUST run the ablation in 7.6 and report the comparison honestly.

4. Repository Layout (exact)

```
network-intrusion-detection/
├── config/
│   ├── config.yaml          # paths, split, training, thresholds, severity
(single source of truth)
│   ├── columns.yaml         # metadata aliases, eligible-feature rules
│   └── label_map.yaml
├── data/
│   ├── raw/cicids2017/      # user-supplied CSVs (git-ignored)
│   ├── interim/             # cleaned per-file Parquet (git-ignored)
│   ├── processed/           # train/val/test Parquet (git-ignored)
│   └── runs/                # saved analysis runs (git-ignored)
├── models/                  # model bundles + FROZEN marker (git-ignored
except .gitkeep)
├── reports/                 # schema_report.json, metrics JSON, figures,
model_card.md
├── notebooks/               # optional EDA only; no logic that the pipeline
depends on
├── src/nids/
│   ├── common/              # config loading, logging, paths, errors
```

```
|   ├── data/                # adapters.py, loader.py, validate.py, clean.py,
split.py
|   ├── features/            # preprocess.py (transformers), selection.py
|   ├── training/            # train.py, tune.py, threshold.py, calibrate.py
|   ├── evaluation/          # metrics.py, report.py, plots.py
|   ├── inference/           # bundle.py, predictor.py, severity.py,
explain.py
|   └── storage/             # run_store.py (files), db.py (optional
PostgreSQL)
├── backend/
|   ├── main.py              # app factory, lifespan model loading
|   ├── deps.py
|   ├── routes/              # health.py, model.py, predict.py, explain.py
|   └── schemas/             # requests.py, responses.py, errors.py
├── frontend/
|   ├── app.py               # st.navigation entry
|   ├── api_client.py
|   ├── theme.py
|   ├── components/          # kpi.py, tables.py, charts.py
|   └── pages/               # 1_overview.py ... 6_analysis_lab.py
├── scripts/                 # run_pipeline.py, score_heldout.py,
freeze_model.py, make_fixture.py
├── tests/                   # unit/, api/, integration/, fixtures/
├── docs/                    # DECISIONS.md, PROGRESS.md, ARCHITECTURE.md
├── .streamlit/config.toml
├── .env.example
├── .gitignore
├── pyproject.toml
├── requirements.txt / requirements-dev.txt / requirements-postgres.txt
└── README.md
```

Justification of deviations from the proposal: the package lives in `src/nids/` (installable, no path hacks); `config/` centralizes tunables; `scripts/` holds thin CLI entry points; `storage/` isolates persistence. No other layers are allowed.

5. Data Pipeline Specification

5.1 Loading (`data/loader.py`).

- `load_raw_file(path) -> DataFrame`. Read with `pandas.read_csv(path, low_memory=False)` trying encodings in order `utf-8`, `cp1252`, `latin-1`; use the first that succeeds and log which one.

- Normalize column names (3.6). If a normalized name appears twice: if the two columns are identical in content, drop the later one; otherwise rename the later one `<name>__dup2`, exclude it from features, and log a warning.
- Add `source_file` (file name) and `row_in_file` (0-based position in the original CSV after header). After concatenating files in sorted file-name order, add a global unique integer `row_id`.
- A file with zero data rows raises `EmptyFileError`. Memory rule: process one file at a time; write cleaned Parquet per file (5.3); only concatenate the cleaned, float32 versions.

5.2 Validation (`data/validate.py`) writes `reports/schema_report.json` and `reports/schema_report.md`. Required content:

1. Files found, detected variant (A/B), row count per file, encoding used.
2. Per column: dtype, missing count, +inf count, -inf count, negative count, non-numeric count, number of unique values, constant flag.
3. Cross-file consistency: the set of normalized column names per file. If sets differ, continue on the intersection only if it is at least 90% of the union (record the dropped columns); otherwise STOP (0.4(c)).
4. Label statistics: raw label strings, normalized strings, mapping to family, counts per file and overall, both before and after cleaning.
5. Exact duplicate count (on eligible features + fine label), overall and per class.
6. Timestamp diagnostics (variant A): parse success rate, min/max date, histogram of hours, fraction of consecutive rows with non-decreasing timestamp per file.
7. Imbalance summary: class proportions and imbalance ratio (majority count / minority count).

Hard assertions (raise, do not continue): Label column exists in every file; at least one numeric feature column remains; no file is mixed variant.

5.3 Cleaning (`data/clean.py`), exact order of operations.

1. Drop rows whose Label is missing, blank, or equal to the literal header text `Label` (repeated header). Count them.
2. Normalize and map labels (3.5). Create `label_fine`, `label_family`, `label_binary`.
3. Coerce every eligible feature column with `pd.to_numeric(errors='coerce')`; count values that became NaN because they were non-numeric.
4. Replace +inf and -inf with NaN. Count per column.

5. Drop rows where ALL feature values are NaN. Do NOT drop rows for isolated NaN (imputation happens later, fitted on train only). Do NOT clip or drop negative values.
6. Cast features to `float32`. After the cast, any value that overflowed to inf is set to NaN and counted.
7. Write `data/interim/<source_file_stem>.parquet`.
8. After all files are cleaned, concatenate and remove exact duplicates (5.4).
9. Write `reports/cleaning_report.json` with counts at every step, per class: `rows_in`, `dropped_bad_label`, `dropped_all_nan`, `nan_from_non_numeric`, `inf_replaced`, `duplicates_removed`, `rows_out`.

Performance requirement: the full dataset (about 2.8 million rows before cleaning, to be verified from the report) MUST be cleanable in under 8 GB peak RAM. If not, process and write by file and use categorical/float32 dtypes.

5.4 Duplicate handling. Duplicate key = all eligible feature columns plus `label_fine`; metadata (IPs, ports, timestamp, Flow ID) is NOT part of the key, because flows that differ only in metadata are behaviourally identical and would leak across splits. Keep the first occurrence in (file name, `row_in_file`) order. Implementation: compute `pandas.util.hash_pandas_object(frame[key_columns], index=False)`, find hash-equal groups, verify exact equality against the first member of each group, and remove only verified duplicates. Rows with identical features but DIFFERENT fine labels are a data-quality conflict: keep the first occurrence only, and write the conflict count and up to 20 example `row_id`s to `cleaning_report.json` under `label_conflicts`.

5.5 Time handling and ordering.

- Splitting NEVER depends on parsed timestamps. Ordering key = (`source_file` sorted by name, `row_in_file`). This is robust to the timestamp format and 12-hour ambiguity issues in 3.4(8). Each dataset file is a recorded session written in approximately chronological order; the validation report's monotonic-fraction statistic documents how true this is.
- Timestamps are parsed for display only: `pd.to_datetime(..., errors='coerce')` trying the formats `%m/%d/%Y %H:%M:%S`, `%m/%d/%Y %H:%M`, `%d/%m/%Y %H:%M:%S`, `%d/%m/%Y %H:%M`, `%Y-%m-%d %H:%M:%S`; choose the single format with the highest parse rate for the whole file set (ties: the order above). Parsed dates outside `config.timestamp.expected_date_range` (default 2017-07-03 to 2017-07-07) become NaT.

- 12-hour ambiguity rule: if the hour histogram contains no value 0 and no value of 13 or higher, and contains hours 8 to 12 together with hours 1 to 7, then hours 1 to 7 are treated as afternoon (add 12 hours) and the metadata flag `timestamp_adjusted=true` is recorded. If the evidence does not match this pattern, do not adjust.
- Timeline mode for the dashboard: `timestamp` if at least 95% of timestamps parse, else `sequence` (use row order position as x-axis, labelled Flow sequence). Uploaded CSVs follow the same rule per run.

5.6 Splitting (`data/split.py`). Row-level random splitting is FORBIDDEN. Algorithm (config: `split.block_rows=10000`, `split.purge_rows=200`, `split.ratios=[0.70,0.15,0.15]`, `split.seed=42`):

1. Within each source file, cut consecutive rows into blocks of `block_rows`; a final block shorter than `block_rows/2` is merged into the previous block. Each block gets a `block_id`.
2. For every block compute counts per `label_family`.
3. Assign whole blocks to train/val/test with a deterministic greedy stratified procedure: shuffle blocks with `seed`; for each block pick the split with the largest weighted deficit, where deficit = sum over families of (target_count_for_split minus current_count) divided by the family's total count; targets are ratio times family total.
4. Purge: for every adjacent pair of blocks in the same file that belong to different splits, mark `purge_rows` rows on each side of the boundary as `purged` and exclude them. Report the number purged per class.
5. Write `data/processed/train.parquet`, `val.parquet`, `test.parquet` and `reports/split_report.json`.

Post-conditions (assert): no `row_id` in two splits; the sum of rows (including purged) equals the cleaned total; the split report lists per-split counts per family and per fine label; `low_support` list = classes with fewer than `config.split.min_per_split` (default 20) rows in any split.

Edge cases: a family with zero rows in TRAIN raises `SplitError` (STOP). A family with zero rows in VAL or TEST is allowed, recorded, excluded from per-class metrics, and displayed in the dashboard as `No samples in this split`. A single-class file (for example Monday, benign only) is allowed.

5.7 Novelty diagnostic (required, not a filter). For each test row compute a near-duplicate key: round each feature to 3 significant digits, hash the vector, and flag the row if the key exists in the training set. Report the near-duplicate rate per class in

`reports/split_report.json` and compute secondary test metrics on the `novel` subset (rows NOT flagged), stored as `test_metrics_novel.json`. This shows how much performance depends on repeated traffic patterns.

5.8 Unseen-attack experiment (SHOULD). Leave-one-family-out for the binary Random Forest: for each attack family F present in train, retrain without F rows, then report the detection rate of F rows in the test split plus the benign FPR. Output `reports/leave_one_family_out.json`. The dashboard shows it as generalization to unseen attack types, never as production performance.

6. Feature Pipeline (`features/`)

6.1 Feature set determination (fit on TRAIN only), in order:

1. Candidates = numeric columns not in the forbidden list of 3.7 (and, per 7.6 outcome, optionally excluding `Destination Port` and `Protocol`).
2. Drop columns that are entirely NaN in train.
3. Drop constant columns (at most one distinct non-NaN value in train).
4. Drop one of each highly correlated pair: Pearson |r| above `features.corr_threshold` (default 0.98), computed on a seeded sample of at most 200,000 train rows after median imputation. Keep the earlier column in column order, drop the later one. Record every dropped column and its partner in `reports/feature_report.json`.
5. Store the final ordered `feature_names` list.

Feature selection MUST NOT use the validation or test splits or labels outside train.

6.2 Transformers (sklearn-compatible, in `features/preprocess.py`).

Class	Behaviour
`FlowFrameSanitizer`	`fit(df)`: runs 6.1 and learns `feature_names`. `transform(df)`: selects `feature_names` by NAME in that order (extra columns ignored, order irrelevant), raises `MissingFeaturesError(missing=[...])` if any is absent, coerces to numeric, replaces +inf/-inf with NaN, returns `float32` array.
`SimpleImputer(strategy='median')`	Fit on train. Medians are stored in the bundle.

Class	Behaviour
`SignedLog1p`	Used only in LR and Isolation Forest pipelines: sign(x) times log1p(abs(x)). Tames heavy-tailed rate and duration features.
`StandardScaler`	Used only after `SignedLog1p` in LR and Isolation Forest pipelines. Tree models (Random Forest, XGBoost) are not scaled.

6.3 Contract. For any cleaned DataFrame `df`, `pipeline.predict_proba(df)` called at inference time MUST equal the value computed in training evaluation (unit test T-PRE-3, tolerance 1e-6). The API and dashboard MUST pass raw feature DataFrames to the saved pipeline and MUST NOT re-implement imputation, scaling, or column selection.

6.4 Dtype and value policy at inference. Numbers and numeric strings are accepted; booleans, other strings, and objects become NaN and are counted in `non_numeric_values`. NaN and null are imputed with train medians and counted in `imputed_feature_count`. Values outside the training range are NOT clipped; they are passed through (tree models are robust) and a flag `out_of_range_feature_count` (values beyond the train min/max for that feature) is returned for transparency.

7. Training Specification

7.1 Targets. Binary: `label_binary` (1 = malicious). Multiclass: `label_family`, encoded with a saved `LabelEncoder` (classes sorted alphabetically, `BENIGN` included).

7.2 Training-set construction. Take `train.parquet`; cap rows per `label_fine` at `train.max_rows_per_class` (default 150,000) by seeded random sampling without replacement; classes below the cap keep all rows. Record effective counts in `reports/train_report.json`. Validation and test are never subsampled. Because capping changes class priors, thresholds are chosen on the natural-prior validation split (8.5), and scores must not be read as probabilities (R8).

7.3 Models.

Model	Task	Specification
Logistic Regression	binary (multiclass optional)	Pipeline: Sanitizer, Imputer, SignedLog1p, StandardScaler, `LogisticRegression(solver='lbfgs', max_iter=1000, C=1.0, class_weight='balanced')`. Role: interpretable baseline.
Random Forest	binary and multiclass	Pipeline: Sanitizer, Imputer, `RandomForestClassifier(n_estimators=200, max_features='sqrt', min_samples_leaf=2, class_weight='balanced_subsample', n_jobs=-1, random_state=42)`. Role: initial primary model.
XGBoost	binary and multiclass	Pipeline: Sanitizer, Imputer, `XGBClassifier(tree_method='hist', n_estimators=500, learning_rate=0.1, max_depth=6, subsample=0.8, colsample_bytree=0.8, early_stopping_rounds=30, eval_metric='aucpr' for binary and 'mlogloss' for multiclass, n_jobs=-1, random_state=42)`. Binary: `scale_pos_weight = n_negative / n_positive` on the capped train set. Multiclass: `sample_weight` from `compute_sample_weight('balanced')`. Early stopping uses the validation split (a small optimistic bias on validation is accepted and documented; the test split stays untouched).
Isolation Forest	binary only	Pipeline: Sanitizer, Imputer, SignedLog1p, StandardScaler, `IsolationForest(n_estimators=200, contamination='auto', random_state=42)` fitted on BENIGN train rows only. Score = negative `score_samples` (higher = more anomalous). Labelled `unsupervised baseline` everywhere.

Random Forest tuning: randomized search over at most 12 configurations drawn from `n_estimators` in [100, 200, 300], `max_depth` in [None, 20, 40], `min_samples_leaf` in [1, 2, 5], each scored on the VALIDATION split (not cross-validation on train, which would reintroduce row-level leakage). Log all configurations and scores in `reports/tuning_rf.json`.

7.4 Combined system output. The binary model plus its operating threshold decides `is_malicious_prediction`. If malicious and a multiclass model exists: `attack_family` = the attack class (excluding BENIGN) with the highest multiclass score; `family_score` = that raw score (not renormalized). If the multiclass argmax over ALL classes is BENIGN while the binary model says malicious, set `family_uncertain = true`. If no multiclass model exists, `attack_family` = `null`.

7.5 Model selection (validation only). Binary: highest PR-AUC; tie-break (difference below 0.002) by higher recall at FPR at most 1%, then by smaller model size. Multiclass: highest macro-F1 over classes with at least `min_per_split` validation rows. The selection and its justification are written to `reports/model_comparison.json` before `models/FROZEN` is created (R4).

7.6 Required ablations (validation only; output `reports/ablation.json`).

1. Full eligible features versus the same set without `Destination Port` and `Protocol`. Decision rule: if removing them lowers validation PR-AUC by less than 0.005, the final feature set EXCLUDES them (lower shortcut risk); otherwise it includes them and the README states this.
2. Full feature set versus the top 20 features by Random Forest importance (informational).

7.7 Overfitting checks. For each model report train versus validation F1 and PR-AUC. If train F1 minus validation F1 exceeds 0.05, add a warning to the model card.

7.8 Reproducibility. Seed 42 everywhere (`config.seed`). Record library versions, Python version, git commit (if available), config snapshot, and data file list hash in the bundle metadata.

7.9 Safeguards. `--fast` mode: `train.max_rows_per_class=20000`, Random Forest `n_estimators=50`, XGBoost `n_estimators=100`, no tuning. Fast-mode artifacts MUST carry `fast_mode=true`, and the dashboard MUST show a visible banner `Fast mode model: not representative`. Training MUST log wall time per model.

7.10 CLI. `python scripts/run_pipeline.py --stage {validate|clean|split|train|evaluate|all} [--fast]`; `python scripts/freeze_model.py --model-id <id>`; `python scripts/score_heldout.py` (scores the held-out test split with metadata and saves it as a run, see 13.2).

8. Evaluation Specification

8.1 Binary metrics (positive class = malicious). Compute at the operating threshold: confusion matrix (TN, FP, FN, TP), precision, recall (TPR), F1, FPR, FNR, specificity, MCC, balanced accuracy, accuracy (always displayed together with the others and never alone). Threshold-independent: ROC-AUC and PR-AUC (`average_precision_score`). Guards: if a denominator is zero or a split has one class, return `null` with a `reason` string (never 0).

8.2 Multiclass metrics. Per-class precision, recall, F1, support; macro and weighted F1; MCC; confusion matrix as counts and row-normalized; one-vs-rest PR-AUC per class with support above zero.

8.3 Per-attack detection rate. For each `label_fine` attack: support, number predicted malicious by the binary model at the threshold, rate, and a 95% Wilson confidence interval. Supports below 30 are flagged `low_support`.

8.4 Benign false-positive analysis. Report FPR with Wilson CI, and the count of benign flows flagged per source file (so false-positive clusters are visible).

8.5 Threshold selection (validation only). Among thresholds from `roc_curve`, choose the one that maximizes recall subject to FPR at most `thresholds.target_fpr` (default 0.01); ties go to the higher threshold. If none satisfies the constraint, use the max-F1 threshold and set `threshold_policy_fallback=true`. Also output a table of recall at FPR 0.1%, 0.5%, 1%, 5%. Isolation Forest uses the same procedure.

8.6 Calibration. v1 evaluates but does not apply calibration: reliability curve (10 quantile bins) and Brier score on validation and test. `score_type` is always `uncalibrated_model_score` in v1. Even a calibrated score would reflect the dataset class prior, not real network prevalence; the UI MUST say so. Applying isotonic calibration is optional (MAY) and requires at least 1,000 positive and 1,000 negative validation samples; if applied, `score_type=calibrated_on_dataset_prior`.

8.7 Files and reporting protocol.

File	Created by	Content
`reports/val_metrics.json`	train stage	validation metrics for every trained model
`reports/model_comparison.json`	train stage	side-by-side validation table, selected model ids, selection reasons
`reports/test_metrics.json`	evaluate stage (after FROZEN)	test metrics for the selected models and for all models (selected one marked)
`reports/test_metrics_novel.json`	evaluate stage	test metrics on the novel subset (5.7)
`reports/calibration.json`	evaluate stage	reliability bins, Brier score
`reports/figures/*.png`	evaluate stage	confusion matrix, PR curve, ROC curve, per-attack detection bars, feature importance
`reports/model_card.md`	evaluate stage	generated from the JSON files only

Every metrics JSON MUST contain: `generated_at`, `split`, `model_id`, `fast_mode`, `n_rows`, `class_counts`, `threshold`, `threshold_policy`, `score_type`, a `binary` object (tp, fp, tn, fn, precision, recall, f1, fpr, fnr, mcc, roc_auc, pr_auc, accuracy), a `per_attack_detection` list, and an optional `multiclass` object. The model card and README results tables are generated by `evaluation/report.py` from these files. Hand-typed numbers are forbidden (R1).

9. Model Bundle (`inference/bundle.py`)

9.1 Layout. `models/<model_id>/bundle.joblib`, `bundle_meta.json`, `bundle.sha256`. `models/ACTIVE` contains the active `model_id`. `models/FROZEN` is created by `scripts/freeze_model.py` and contains model id, timestamp and the SHA-256 of `reports/model_comparison.json`.

9.2 `ModelBundle` fields. `binary_pipeline`, `multiclass_pipeline` (nullable), `label_encoder`, `feature_names`, `feature_medians`, `feature_ranges` (train min/max), `threshold`, `threshold_policy`, `threshold_policy_fallback`, `family_classes`, `severity_config` (snapshot of 12.2), `background_sample` (at most 200 train rows, optional, for SHAP), `schema_version` (=1), `created_at`, `library_versions`, `dataset_info` (name, variant, files hash), `metrics_refs` (relative report paths), `fast_mode`, `score_type`.

9.3 Loading rules. Verify the SHA-256 before `joblib.load`; on mismatch refuse to start with a clear error. Compare saved and installed versions of scikit-learn, xgboost, numpy and pandas; if different, log WARNING and expose `version_mismatch=true` in `/model/info`. Refuse an unsupported `schema_version`. SECURITY: joblib/pickle executes code on load; load ONLY from the configured `MODEL_DIR`, never from a request-supplied or uploaded path.

10. Inference Module (`inference/predictor.py`)

10.1 API of the module. `Predictor.from_dir(model_dir) -> Predictor`; `Predictor.predict_frame(df, metadata=None, include_family=True) -> DataFrame`; `Predictor.info() -> dict`. Both the FastAPI backend and `scripts/score_heldout.py` call this class; no other code path may load the bundle.

10.2 Steps of `predict_frame`.

1. Normalize incoming column names (3.6). Check that all `feature_names` exist; otherwise raise `MissingFeaturesError`.
2. Compute the binary score with a helper `positive_score(pipeline, X)`: `predict_proba[:, 1]` for classifiers; negative `score_samples` for Isolation Forest.
3. `is_malicious_prediction = score >= threshold`.
4. If `include_family` and a multiclass pipeline exists, compute class scores for all rows (7.4 rules) and fill `attack_family`, `family_score`, `family_uncertain` for malicious rows only; benign rows get nulls.
5. Compute `severity` (12.2) for malicious rows; null for benign rows.
6. Compute transparency flags per row: `imputed_feature_count`, `out_of_range_feature_count`, `non_numeric_values`.
7. Return columns in this fixed order: `row_index`, `score`, `score_type`, `threshold`, `is_malicious_prediction`, `attack_family`, `family_score`, `family_uncertain`, `severity`, `imputed_feature_count`, `out_of_range_feature_count`,

`non_numeric_values`, followed by any passed-through metadata columns.

10.3 Score semantics (document in `/model/info` as `score_description`).

Algorithm	Score	Range
Random Forest	mean of tree class-probability estimates (vote fraction)	0 to 1
XGBoost	predicted positive-class probability, uncalibrated	0 to 1
Logistic Regression	sigmoid output of a class-weighted model, uncalibrated	0 to 1
Isolation Forest	anomaly score (higher = more anomalous)	unbounded

The UI label for the score is `Model score` (never `Probability`, never `Confidence in attack`). A short tooltip explains: this number ranks flows by how attack-like the model finds them; it is not the chance that an attack occurred.

10.4 Performance and resource rules. Predict in internal chunks of 50,000 rows. After loading, set `n_jobs=1` on tree ensembles for serving (single-row requests must not spawn thread pools); batch scoring through `score_heldout.py` MAY use `n_jobs=-1` via `INFERENCE_N_JOBS`. Target latency on a laptop CPU: `/predict` under 300 ms; `/predict/batch` with 5,000 rows under 10 seconds for Random Forest with 200 trees (record the measured values in the README; do not claim them before measuring).

10.5 Edge cases. Zero rows: raise `EmptyInputError`. All-NaN row: allowed (fully imputed) but `imputed_feature_count == len(feature_names)` triggers warning `row_fully_imputed` in the response. Extra columns: ignored and listed once per request in `ignored_fields`. Column order differences: irrelevant.

11. Backend Specification (FastAPI)

11.1 Application. `backend/main.py` exposes `app = create_app()`. A lifespan handler loads the `Predictor` once into `app.state.predictor`. If the bundle is missing or fails verification, the app still starts in degraded mode: `/health` reports `degraded`, and prediction endpoints return 503 `model_not_loaded`. Endpoints that do CPU-bound work MUST be declared with plain `def` (not `async def`) so FastAPI runs them in its thread pool. The API is stateless: it writes nothing to disk or database.

11.2 Settings (`pydantic-settings` , environment variables, defaults shown).

Variable	Default	Meaning
`MODEL_DIR`	`models`	Folder containing model bundles and `ACTIVE`
`API_HOST` / `API_PORT`	`127.0.0.1` / `8000`	Bind address (never `0.0.0.0` by default)
`MAX_BATCH_ROWS`	`5000`	Max flows per batch request
`MAX_REQUEST_BYTES`	`50000000`	Reject larger bodies with 413
`CORS_ORIGINS`	`http://localhost:8501`	Comma-separated allow-list
`API_KEY`	empty	If set, all endpoints except `/health` require header `X-API-Key`
`LOG_LEVEL`	`INFO`	Logging level
`INFERENCE_N_JOBS`	`1`	Threads for tree prediction

11.3 Endpoints.

Method and path	Request	Success response (200)
GET `/health`	none	`status` (`ok` or `degraded`), `model_loaded` , `model_id` , `api_version` , `uptime_seconds` , `time_utc`

Method and path	Request	Success response (200)
GET `/model/info`	none	`model_id`, `created_at`, `dataset_info`, `fast_mode`, `score_type`, `score_description`, `threshold`, `threshold_policy`, `threshold_policy_fallback`, `feature_count`, `feature_names`, `classes`, `algorithms` (binary and multiclass), `metrics` (validation and, if present, test summaries read from the reports; `null` when missing), `library_versions`, `version_mismatch`, `severity_config`, `limitations` (list of strings, includes the 2017 lab-data caveat)
POST `/predict`	`features` (object name to number or null, required), `metadata` (optional), `options.include_family` (bool, default true)	one `PredictionResult` plus `ignored_fields`, `warnings`
POST `/predict/batch`	`flows` (list of objects with `features` and optional `metadata`), 1 to `MAX_BATCH_ROWS` items, `options`	`model_id`, `n_rows`, `n_malicious`, `results` (list of `PredictionResult`, index-aligned with the request), `ignored_fields`, `warnings` (aggregate counts)
POST `/explain`	same as `/predict` plus `top_k` (default 10, max 30)	`prediction` (a `PredictionResult`), `method` (`shap_tree`), `base_value`, `contributions` (list of `feature`, `value`, `shap_value`, sorted by absolute shap value)

11.4 Metadata object (all optional). `flow_id` (string, at most 128 chars), `source_ip` and `destination_ip` (valid IPv4 or IPv6 verified with Python `ipaddress`), `source_port` and `destination_port` (integer 0 to 65535), `protocol` (integer 0 to 255 or string of at most 16

chars), `timestamp` (ISO 8601 string). Metadata is passed through to the response and is never given to the model.

11.5 Feature validation rules (Pydantic v2 plus a validator).

1. Keys are normalized (strip, collapse whitespace). Two keys that collide after normalization: 422 `duplicate_feature_key`.
2. Every name in `feature_names` must be present: otherwise 422 `missing_features` listing up to 20 missing names plus the total count.
3. Values: JSON numbers or null. Strings, booleans, arrays, objects: 422 `invalid_feature_type`. Non-finite tokens (NaN, Infinity) are rejected at parse time: 422 `non_finite_value`. Absolute value above 1e15: 422 `value_out_of_bounds`. Null is accepted and imputed.
4. More than 500 keys in one flow: 422 `too_many_features`.
5. Unknown keys: ignored and echoed in `ignored_fields` (never an error).
6. Batch: validation is all-or-nothing; the error `details` list the first 20 offending `row_index` values with their error codes.

11.6 Error format (every non-2xx response). JSON object with `error.code`, `error.message`, `error.details` (list or null), `error.request_id`. Every response carries header `X-Request-ID` (generated UUID4 if the client did not send one). Code table:

HTTP	`error.code`	When
401	`invalid_api_key`	`API_KEY` set and header missing or wrong
413	`payload_too_large`	body over `MAX_REQUEST_BYTES`
422	`validation_error` and the specific codes in 11.5	invalid input
422	`empty_batch` / `batch_too_large`	batch size outside 1 to `MAX_BATCH_ROWS`
503	`model_not_loaded`	degraded mode
501	`explain_unavailable`	selected model not tree-based, or SHAP not installed
500	`internal_error`	any unexpected exception; log the stack trace server-side, return only the `request_id` to the client

FastAPI's default 422 body MUST be replaced by this format through exception handlers.

11.7 Logging. One JSON-lines log record per request: `request_id`, method, path, status, duration_ms, `n_rows` (batch). Feature values and IP addresses MUST NOT be logged at INFO level.

11.8 Run command. `uvicorn backend.main:app --host 127.0.0.1 --port 8000` (add `--reload` only in development). OpenAPI docs at `/docs`.

12. Alert and Severity Semantics

12.1 Definition. An alert is a flow with `is_malicious_prediction = true`. An alert is a model prediction, NOT a confirmed attack. This sentence (or a close paraphrase) MUST appear on the Alerts and Investigation pages.

12.2 Severity (heuristic triage category, documented and configurable in `config.yaml` under `severity`). Severity = function of attack-family impact and score tier.

- Score tier edges are computed ONLY on the validation split: take validation flows predicted malicious, compute the 33rd and 66th percentiles of their scores, store them in the bundle as `score_tier_edges`. Tier `marginal` = score below the first edge (value

1); `moderate` = between edges (value 2); `strong` = at or above the second edge (value 3). This works for any score scale.

- Family impact values: Bot 3, RareOther 3, WebAttack 3, BruteForce 2, DoS 2, DDoS 2, PortScan 1. Unknown family (no multiclass model, or `family_uncertain = true`) uses impact 2.
- Severity from the sum (impact plus tier value): sum 6 = `Critical`; 5 = `High`; 4 = `Medium`; 2 or 3 = `Low`.
- Colours: Critical and High red, Medium amber, Low yellow-green; benign and healthy status green.
- The UI shows a tooltip: severity is a triage heuristic derived from predicted family and model score; it is not a measured risk level.

12.3 Benign predictions have `severity = null`.

13. Storage

13.1 Default: file-based `RunStore` (`storage/run_store.py`). A run is an analysis of one dataset or upload. Directory `data/runs/<run_id>/` with `run_id = <UTC YYYYMMDDTHHMMSSZ>_<6 random hex>`. Files:

File	Content
`predictions.parquet`	the output columns of 10.2, metadata columns present in the input (canonical names), and `true_label_fine`, `true_label_family` when valid labels exist
`features.parquet`	the `feature_names` columns (float32) with `row_index`, used by the Investigation page
`run_meta.json`	`run_id`, `created_at`, `kind`, `source_name` (sanitized), `source_sha256`, `model_id`, `threshold`, `n_rows`, `n_malicious`, `timeline_mode`, `has_labels`, `warnings`

`kind` is one of `USER_UPLOAD`, `HELDOUT_TEST_REPLAY`, `SYNTHETIC_DEMO` (R7). Uploaded file names are reduced to the base name, characters outside `A-Za-z0-9._-` replaced with `_`, truncated to 100 characters. Maximum rows per run: `MAX_RUN_ROWS = 1,000,000`; larger uploads are rejected with a clear message before scoring.

13.2 Held-out replay (`scripts/score_heldout.py`). Scores the test split (with metadata when variant A) by calling `Predictor` directly, and saves a run of kind `HELDOUT_TEST_REPLAY` including true labels. The dashboard banner for this run: `Offline replay of held-out CIC-IDS2017 test flows. This is recorded data, not live traffic.` This run is the default content of the dashboard once the pipeline has completed.

13.3 Uploads that contain labels. If an uploaded CSV has a `Label` column whose normalized values ALL map through `label_map.yaml` , store true labels and enable the optional `Metrics on this file` panel (computed with the same functions as Section 8, labelled `computed on uploaded file`). If any label is unmapped, ignore labels for the whole file and warn. If the file name matches one of the files in `dataset_info.files` , show the warning `This file may contain training data; metrics will be optimistic.`

13.4 Optional PostgreSQL (`storage/db.py` , enabled by `STORAGE_BACKEND=postgres` and `DATABASE_URL`). SQLAlchemy 2.0 Core, tables created with `create_all` (no migration tool).

Table	Columns
`analysis_runs`	`run_id` text primary key, `created_at` timestamptz, `kind` text, `model_id` text, `source_name` text, `n_rows` int, `n_malicious` int, `meta` jsonb
`flow_predictions`	`id` bigserial primary key, `run_id` text references `analysis_runs` on delete cascade, `row_index` int, `flow_id` text, `source_ip` inet, `destination_ip` inet, `source_port` int, `destination_port` int, `protocol` text, `event_time` timestamptz, `score` double precision, `is_malicious` boolean, `attack_family` text, `family_score` double precision, `family_uncertain` boolean, `severity` text, `true_label_fine` text

Indexes: (`run_id` , `is_malicious` , `severity`), (`run_id` , `event_time`), (`run_id` , `attack_family`). Insert in chunks of 10,000 rows inside one transaction per run. Features remain in `features.parquet` (hybrid design). Rules: parameterized statements only; if

the connection fails, log the error, fall back to the file backend, and show a dashboard warning (never crash). PostgreSQL tests are marked `@pytest.mark.postgres` and skipped unless `TEST_DATABASE_URL` is set.

14. Frontend Specification (Streamlit)

14.1 Structure. `frontend/app.py` sets `st.set_page_config(page_title='NIDS SOC Dashboard', layout='wide')` and builds navigation with `st.navigation`. Pages (file, title): `1_overview.py` Security Overview; `2_alerts.py` Threat Alerts; `3_investigation.py` Traffic Investigation; `4_network.py` Network Activity; `5_performance.py` Model Performance; `6_analysis_lab.py` Traffic Analysis Lab. Shared code: `api_client.py` (HTTP), `data.py` (cached loaders), `theme.py` (colours, CSS, Plotly template), `components/` (KPI card, tables, chart helpers). Pages contain layout only; no ML logic.

14.2 Theme (`.streamlit/config.toml` and `theme.py`).

Token	Value
Streamlit base	`dark`
Page background	`#0b1220`
Panel / card background	`#111a2e`
Border (1 px, subtle)	`#1f2a44`
Text / muted text	`#e5e7eb` / `#94a3b8`
Accent (links, selection)	`#38bdf8`
Critical / High	`#ef4444` / `#f97316`
Medium (amber)	`#f59e0b`
Low	`#84cc16`
Benign / healthy / API up	`#22c55e`

Typography: Streamlit sans-serif for text; monospace for IP addresses, ports and feature names. The Plotly template uses transparent paper and plot backgrounds, grid colour `#1f2a44`, and the colours above. Severity is NEVER conveyed by colour alone: also show the text label.

14.3 Global sidebar and banners (on every page).

1. Run selector listing saved runs newest first (label: kind, source name, created time, row count); default: newest `HELDOUT_TEST_REPLAY`, else newest run.
2. API status pill (green `API online`, red `API offline`) from `GET /health`, cached for 10 s; also shows `model_id`.
3. Banners: run kind (`DEMO` / `SYNTHETIC` / `UPLOAD` / `Offline replay of recorded data`), `Fast mode model: not representative` when `fast_mode`, `Timeline: flow sequence (no usable timestamps)` when applicable, `Version mismatch` warning when set.
4. Footer: `Predictions are model outputs, not confirmed attacks. Scores are uncalibrated. Trained on lab-generated 2017 data; results may not transfer to other networks.`

14.4 Empty and failure states (required). No runs yet: show the exact commands from the README to run the pipeline. API offline: pages 1 to 5 keep working from stored runs; page 6 and the `Explain` button show `Start the API: uvicorn backend.main:app` and are disabled. Missing report files: `Not yet generated. Run the pipeline.` An exception inside a page is caught and shown as a red error panel with the `request_id` if available, never a raw traceback.

14.5 Performance rules. Loaders use `st.cache_data` keyed by `run_id` plus file modification time. Aggregate with pandas BEFORE plotting. No scatter or table rendering above 5,000 points or rows at once (paginate tables at 100 rows; sample or aggregate charts). Time bucketing chooses the smallest bucket from 1 min, 5 min, 15 min, 1 h, 6 h, 1 day that yields at most 500 buckets.

14.6 Page A: Security Overview.

- KPI row: total flows analysed; predicted malicious (count and percent); predicted benign; alerts by severity (four coloured counters); active model id and threshold.
- Threat distribution: donut chart of `attack_family` among alerts (falls back to severity when no family exists).
- Traffic over time: stacked area of benign versus malicious predicted flows per time bucket (or per sequence bucket).
- Recent alerts: 20 most recent (by timestamp, else by sequence position descending) with columns Time, Source IP, Destination IP, Family, Score, Severity; absent columns are omitted.

- If true labels exist (replay or labelled upload): a separate panel titled `Ground-truth comparison (dataset labels)` with confusion counts. This panel must be visually separate from the prediction KPIs.

14.7 Page B: Threat Alerts.

- Table of alerts only, columns: Time, Source IP, Destination IP, Destination Port, Protocol, Predicted family, Model score, Severity, Row. Show only columns that exist; show a note listing hidden ones.
- Filters: severity (multiselect), family (multiselect), time range (only when timeline is `timestamp`), minimum score slider, IP substring search (plain substring match, no regex), sort selector.
- Selecting a row sets `st.session_state['selected_row']` and offers `Investigate this flow` (switch to page C).
- Download the filtered alerts as CSV (sanitized, 14.12). An expander `Definitions` explains alert, model score and severity (Section 10.3 and 12).

14.8 Page C: Traffic Investigation.

- Flow selector: integer input `Row` validated against the run size, prefilled from the Alerts page.
- Prediction card: score, threshold, margin to threshold, predicted family and family score, severity, `family_uncertain` badge, transparency counts (imputed, out of range). Banner: `Model prediction, not a confirmed attack.`
- If ground truth exists: a separate card `Dataset label (ground truth)`.
- Metadata table (IPs, ports, protocol, timestamp when present).
- Feature table: feature, value, training median, ratio to median (blank when median is 0), highlighted when out of training range. Search box over feature names.
- `Explain this flow` button calls `POST /explain`; shows a horizontal bar chart of the top 10 contributions (red = pushes toward malicious, green = pushes toward benign) with units from the response. Handle 501/503 by showing the global feature-importance chart from `reports/feature_importance.json` labelled `Global importance (not specific to this flow)`.

14.9 Page D: Network Activity.

- Always available: protocol distribution (bar), top 15 destination ports (bar, benign versus malicious stacked), flow count over time, and a histogram of one user-selected feature (log-scaled x axis when values are positive and span more than 3 orders of

magnitude) comparing predicted-benign versus predicted-malicious.

- Requires IP columns: Sankey of the top 25 source to destination IP pairs by flow count with the malicious share shown through link colour; top 15 sources by alert count.
- `Descriptive patterns of flagged flows` table: per source IP, alert count, number of distinct destination ports, number of distinct destination IPs. Caption: `Descriptive aggregates of model-flagged flows. These are not additional detections.`
- Optional network graph (toggle, off by default): `networkx` spring layout with `seed=42` on the top 100 source-destination edges by alert count, at most 150 nodes. Skip with a message when there are no edges.
- When a required column is missing, replace the chart by `Requires Source IP and Destination IP columns`. No external lookups of any kind (no geolocation, no reverse DNS, R6).

14.10 Page E: Model Performance. Data source: only the JSON files and figures in `reports/`. Content: model selector (default: selected model); KPI row (precision, recall, F1, FPR, PR-AUC, ROC-AUC, with 95% CI where available and the operating threshold); confusion matrix heatmap with counts/normalized toggle; recall-versus-FPR curve (from stored curve points, at most 500); model comparison table for validation and test with the selected model highlighted; multiclass per-class table (precision, recall, F1, support, `low_support` badge) and confusion matrix; per-attack detection-rate bars with CI; test versus `novel subset` metrics side by side; calibration reliability plot with caption `Scores are not calibrated probabilities`; leave-one-family-out table; ablation table; a `Limitations` panel. Never recompute metrics in the frontend.

14.11 Page F: Traffic Analysis Lab (workflow).

1. Upload: `st.file_uploader(type=['csv'])`; `server.maxUploadSize = 200` (MB) in config; also check size in code.
2. Parse: use the shared loader normalization (encoding chain, column-name normalization, duplicate columns). If the file parses to a single column and contains `;` or a tab in the header, retry with that separator. Decimal commas are not supported (state this in the UI help text). Zero data rows: error `The file has a header but no rows`. More than `MAX_RUN_ROWS` rows: error with the limit.
3. Validation panel: row count; features found versus required; missing required features (list, up to 30 names plus count); extra columns; NaN and infinity counts; exact duplicate rows; label column found and mappable; timestamp parse rate. Status: `PASS`

(all required features present), `WARN` (extra columns, NaN/inf, duplicates, unmapped labels), `FAIL` (missing required features: analysis disabled; offer a downloadable header-only template CSV built from `feature_names`).

4. Analyse (button): sanitize values (+inf, -inf, NaN to null), split into chunks of 5,000 rows, call `POST /predict/batch` sequentially with a progress bar. HTTP timeout 120 s; up to 2 retries (waits 1 s then 3 s) on connection errors and 5xx; stop immediately on 4xx and show `error.code`, `error.message` and `request_id`. All-or-nothing: save the run only when every chunk succeeded. Guard against duplicate runs on Streamlit reruns using a hash of file content plus model id kept in `st.session_state`.
5. Results: prediction summary (counts by severity and family), two charts, `Open in dashboard` (sets the active run), and two downloads: `Predictions only` and `Original columns plus predictions`.

14.12 CSV download safety (formula injection). For every string cell in a downloaded CSV, if it starts with `=`, `+`, `-`, `@`, a tab, or a carriage return, prefix it with a single quote. Numeric dtype columns are not modified. Encoding UTF-8. Downloaded prediction files include a final comment-free header row only (no extra metadata rows) so they remain valid CSVs; metadata (model id, threshold, generation time, run kind) is delivered as a second small downloadable `run_meta.json`.

14.13 Run commands. `API_URL` defaults to `http://127.0.0.1:8000`; `API_KEY` (if set) is sent as `X-API-Key`. `streamlit run frontend/app.py --server.port 8501 --server.address 127.0.0.1`. `.streamlit/config.toml` sets `browser.gatherUsageStats = false`.

15. Explainability (`inference/explain.py`)

15.1 Local explanations. `explain_flow(predictor, row, top_k)` supports: Random Forest and XGBoost through `shap.TreeExplainer` applied to the array produced by all pipeline steps before the final estimator; Logistic Regression through exact linear contributions (coefficient times standardized value, method `linear_contrib`). Isolation Forest is unsupported (501). The explainer is built lazily once and guarded by a lock.

15.2 SHAP output quirks the implementation MUST handle.

- Depending on the SHAP version, a Random Forest classifier returns either a list of per-class arrays or one 3-dimensional array (rows, features, classes). Normalize to the positive-class (malicious) vector.

- XGBoost binary returns contributions in log-odds units; Random Forest in probability units. Return a `units` field (`probability` or `log_odds`) and show it in the UI.
- `expected_value` may be a scalar or an array; select the positive-class entry.
- Use `check_additivity=False` for Random Forest (float32 versus float64 rounding can raise spurious additivity errors); log when the sum differs from the model output by more than 1e-3.
- Explain exactly one row per call; reject more with 422.

15.3 Global importance (`reports/feature_importance.json`). Native importances (impurity-based for Random Forest, gain for XGBoost) and permutation importance on a stratified validation sample of at most 20,000 rows (3 repeats, scoring `average_precision`), both labelled with their method. A SHAP summary figure on at most 2,000 validation rows is OPTIONAL.

15.4 Wording rule. Explanations describe how the model reached its output, not why an attack happened. The UI shows: `Contributions explain the model, not the cause of the traffic.`

16. Testing Specification

16.1 Principles. Tests MUST run without the real dataset. A synthetic fixture generator (`tests/fixtures/make_fixture.py`) produces a small DataFrame (about 3,000 rows) with variant-A metadata columns, about 20 numeric feature columns with fixture-specific names, two or three attack families plus benign (separable Gaussian clusters with overlap and noise), and injected defects: leading-space column names, a duplicated column name, +inf and NaN values, negative values, exact duplicate rows, a label with a non-ASCII dash, a repeated header row, and a blank-label row. Everything derived from it is labelled `SYNTHETIC` (R7) and is never used to report model performance. A session-scoped `conftest.py` fixture trains a tiny Random Forest on this data and writes a bundle to a temporary directory for API tests.

16.2 Required tests.

ID	Assertion
T-LOAD-1	Column names normalized (strip, collapse spaces); duplicate column handled per 5.1

ID	Assertion
T-LOAD-2	A cp1252-encoded file with a non-ASCII dash in a label loads and the label normalizes to `-` form
T-LOAD-3	Repeated header row and blank-label row are dropped and counted
T-LOAD-4	Header-only file raises `EmptyFileError`
T-CLEAN-1	+inf and -inf become NaN and are counted per column
T-CLEAN-2	Negative values are preserved
T-CLEAN-3	An unmapped label raises `UnmappedLabelError` listing every offender
T-CLEAN-4	Exact duplicates removed ignoring metadata; label conflicts reported
T-CLEAN-5	All-NaN feature rows are dropped; isolated NaN is kept
T-SPLIT-1	No `row_id` appears in more than one split
T-SPLIT-2	Same seed gives identical splits; a different seed gives a different split
T-SPLIT-3	Every `block_id` belongs to exactly one split
T-SPLIT-4	Rows within `purge_rows` of a cross-split boundary are excluded
T-SPLIT-5	A family absent from train raises `SplitError`
T-SPLIT-6	Shuffling the Timestamp column does not change the split
T-PRE-1	Sanitizer selects by name, ignores column order and extra columns
T-PRE-2	Missing features raise `MissingFeaturesError` listing them
T-PRE-3	Training-path and inference-path predictions are equal within 1e-6

ID	Assertion
T-PRE-4	Imputer medians come from train only (train and validation built with different medians)
T-PRE-5	Constant-column and correlated-pair dropping is deterministic and uses train only (changing validation data does not change `feature_names`)
T-EVAL-1	Metrics equal hand-computed values for a known confusion matrix
T-EVAL-2	Zero denominators return `null` with a reason, not 0
T-EVAL-3	Chosen threshold satisfies FPR at most target on validation; impossible target sets `threshold_policy_fallback`
T-EVAL-4	Wilson interval for 0 successes in 10 trials has upper bound close to 0.2775
T-EVAL-5	Final evaluation refuses without `models/FROZEN` and refuses to overwrite `test_metrics.json` without `--force`
T-BUNDLE-1	Save then load gives identical predictions
T-BUNDLE-2	Corrupted `bundle.joblib` (hash mismatch) is refused
T-BUNDLE-3	Unsupported `schema_version` is refused
T-SEV-1	Severity matrix: impact 3 plus tier 3 gives Critical; impact 1 plus tier 1 gives Low; unknown family uses impact 2; benign gives null
T-API-1	`/health` returns ok with a model and degraded without
T-API-2	`/model/info` contains all documented fields
T-API-3	`/predict` happy path returns documented fields with `score_type = uncalibrated_model_score`
T-API-4	Missing features give 422 `missing_features` with the list and count

ID	Assertion
T-API-5	String, boolean and array values give 422 `invalid_feature_type`
T-API-6	NaN and Infinity tokens give 422 `non_finite_value`
T-API-7	Null values are accepted and counted in `imputed_feature_count`
T-API-8	`/predict/batch` results are index-aligned (`row_index`) with the request
T-API-9	Empty batch gives 422 `empty_batch` ; oversized batch gives 422 `batch_too_large`
T-API-10	Body above `MAX_REQUEST_BYTES` gives 413
T-API-11	`API_KEY` enforced on all endpoints except `/health`
T-API-12	Degraded mode returns 503 `model_not_loaded` on prediction endpoints
T-API-13	Every error uses the documented format and `X-Request-ID` header is present
T-API-14	Keys colliding after normalization give 422 `duplicate_feature_key`
T-API-15	Unknown keys are echoed in `ignored_fields`
T-API-16	Invalid IP in metadata gives 422
T-API-17	`/explain` returns contributions sorted by absolute value, or 501 for unsupported models
T-STORE-1	`RunStore` round trip; file name `../../x.csv` is sanitized to a safe base name
T-STORE-2	CSV formula-injection sanitizer prefixes dangerous string cells only
T-STORE-3	A run above `MAX_RUN_ROWS` is rejected before scoring

ID	Assertion
T-FE-1	`api_client` retries on 5xx and connection errors, not on 4xx (mocked HTTP)
T-FE-2	Chunking converts +inf, -inf, NaN to null and yields chunks of at most 5,000 rows
T-FE-3	(SHOULD) `streamlit.testing.v1.AppTest` smoke test of each page against a fixture run
T-INT-1	End-to-end on the synthetic fixture: validate, clean, split, train, freeze, evaluate, API prediction, run store, no exceptions

16.3 Commands and thresholds. `pytest -q`; `pytest --cov=nids --cov=backend --cov-report=term-missing` with line coverage of at least 80% on `src/nids` and `backend`; `pytest -m 'not postgres'`; `ruff check .`; `black --check .`. Optional GitHub Actions workflow runs the same on Python 3.11 using only the synthetic fixture.

17. Implementation Roadmap (Phases, Acceptance Checks, Difficulty)

Difficulty is 1 (easy) to 5 (hard). Effort is a rough guide for a part-time student; the agent's own time will differ. A phase is complete only when ALL its acceptance checks pass (R9). After each phase append the result to `docs/PROGRESS.md`.

Phase	Goal	Difficulty	Effort
1	Scaffold, config, dataset loading, validation report	2	1 to 2 days
2	Cleaning, leakage-aware split, baseline models	3	3 to 4 days
3	Improved models, tuning, ablations, thresholds, frozen evaluation	4	3 to 5 days
4	Saved bundle, predictor, severity	3	1 to 2 days
5	FastAPI backend	3	2 to 3 days
6	Streamlit SOC dashboard	4	4 to 6 days
7	Explainability, downloads, optional PostgreSQL	4	3 to 5 days

Phase	Goal	Difficulty	Effort
8	Tests completion, documentation, GitHub and demo preparation	2	2 to 3 days

Phase 1: Scaffold and dataset understanding.

- Tasks: create the layout of Section 4; `pyproject.toml`; requirements files; `config/*.yaml` (Appendix A and D); `.gitignore` (data, models, reports/figures, `.env`, `.venv`, `data/runs`); `data/raw/cicids2017/README.txt`; implement `adapters.py`, `loader.py`, `validate.py`; fixture generator.
- Acceptance: `pip install -e .` works in a clean venv; `pytest tests/unit -k LOAD` passes; with the real dataset present, `python scripts/run_pipeline.py --stage validate` writes `reports/schema_report.json` and `.md`; with no dataset, it exits with the 0.4(a) message and exit code 2; `docs/DECISIONS.md` records every deviation.

Phase 2: Cleaning, splitting, baselines.

- Tasks: `clean.py`, `split.py`, `preprocess.py`, `selection.py`; Logistic Regression and Random Forest binary baselines at the default 0.5 score threshold; optional EDA notebook (read-only use of Parquet).
- Acceptance: tests T-CLEAN, T-SPLIT, T-PRE pass; `reports/cleaning_report.json`, `split_report.json`, `feature_report.json` exist; no family has zero train rows; baseline validation metrics JSON contains every field required in 8.7 and the train versus validation gap; the pipeline runs end to end in `--fast` mode in under 10 minutes on a laptop.

Phase 3: Improved models and defensible evaluation.

- Tasks: XGBoost, Isolation Forest, multiclass Random Forest and XGBoost; Random Forest tuning; ablations (7.6); threshold selection (8.5); per-attack detection with Wilson CIs; calibration diagnostics; novelty diagnostic (5.7); leave-one-family-out (5.8); model selection (7.5); `freeze_model.py`; final test evaluation.
- Acceptance: tests T-EVAL pass; `model_comparison.json` records selection reasons; `models/FROZEN` exists BEFORE `test_metrics.json` is written; `test_metrics.json` is written exactly once; the model card is generated and contains no hand-typed numbers (verified by a test that regenerates it and diffs).

Phase 4: Bundle and inference.

- Tasks: `bundle.py`, `predictor.py`, `severity.py`, `score_heldout.py`.

- Acceptance: tests T-BUNDLE, T-SEV, T-PRE-3 pass; predictions from `Predictor` on 1,000 validation rows equal the training-time predictions within 1e-6; the held-out replay run is created.

Phase 5: Backend.

- Tasks: app factory, settings, routes, schemas, error handlers, request-id middleware, size limit, API key, logging; `scripts/example_request.py` that builds a real request from the first validation row and prints the response.
- Acceptance: all T-API tests pass; `uvicorn backend.main:app` starts with a model and in degraded mode without one; `/docs` lists all endpoints; the measured latency figures are recorded in the README (10.4).

Phase 6: Dashboard.

- Tasks: theme, shared components, six pages, API client with retry, Analysis Lab workflow.
- Acceptance: T-FE tests pass; manual checklist below completed and recorded in `docs/PROGRESS.md`.

Manual check	Expected
Start without runs	Empty state with commands
Replay run selected	All six pages render; replay banner visible
API stopped	Pages 1 to 5 work; page 6 disabled with instruction
Upload valid fixture CSV	PASS, analysis completes, run saved, downloads work
Upload CSV missing features	FAIL panel, missing list, template download
Upload CSV with NaN, inf, extra columns	WARN panel, analysis completes
Upload header-only CSV	Clear error
Run with zero alerts	Friendly empty states, no chart crashes
Run without IP or timestamp columns	Dependent panels replaced by explanatory messages

Phase 7: Explainability, downloads, optional database.

- Tasks: `explain.py`, `/explain`, Investigation SHAP panel, global importance, filtered downloads with injection sanitizing, optional PostgreSQL backend with fallback.
- Acceptance: T-API-17 and T-STORE pass; PostgreSQL tests pass when `TEST_DATABASE_URL` is set and are skipped otherwise; app works with `STORAGE_BACKEND=files` and no PostgreSQL installed.

Phase 8: Quality, documentation, release.

- Tasks: complete the test table; coverage at least 80%; README (Appendix C); `docs/ARCHITECTURE.md` (data flow diagram described in text plus a Mermaid diagram); model card; dataset citation; `requirements.lock`; verify `git ls-files` contains no dataset, model, or run files and no file above 5 MB; LICENSE file (ask the human which licence; default MIT only if confirmed).
- Acceptance: on a fresh clone, a fresh venv, `pip install -r requirements-dev.txt && pip install -e . && pytest -q` passes using only the synthetic fixture; the README setup and run commands are executed verbatim and work; every number in the README results section is generated from `reports/*.json` by `evaluation/report.py`.

18. Security, Authorisation and Ethics

1. **Authorised data only.** Use only the publicly released CIC-IDS2017 dataset under its published terms. The README MUST cite it (Sharafaldin, Lashkari, Ghorbani, 'Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization', ICISSP 2018) and follow any citation or licence terms stated on the official page, which the agent must read and summarise in `docs/DATASET.md` rather than assume.
2. **No live capture or scanning.** v1 contains no packet capture, no scanning, no attack generation code, and no stubs for them. A future live-monitoring feature requires, before any code: written permission from the network owner, a legal review of interception rules, data minimisation (for example hashing or truncating IPs), access control, retention limits, and an isolated environment. Put this paragraph in the README under `Future work`.
3. **Privacy of uploads.** IP addresses are personal data in many jurisdictions. Uploaded files are processed locally, stored only under `data/runs/`, never sent to third-party services, and never logged at INFO level. Streamlit telemetry is disabled. The README tells users to upload only data they are authorised to analyse.

4. **API hardening.** Bind to 127.0.0.1 by default; optional API key; CORS allow-list; request size and batch size limits; strict Pydantic validation; no `eval` / `exec`; no endpoint accepts a file path; the model directory is fixed by configuration.
5. **Pickle risk.** Model bundles are Python pickles: load only trusted files from `MODEL_DIR`, verify SHA-256 (9.3), and never expose an upload path for bundles.
6. **Output safety.** Do not use `unsafe_allow_html` with any string derived from data; if HTML is needed, escape values. Sanitize CSV downloads (14.12) and uploaded file names (13.1).
7. **Secrets.** `.env` is git-ignored; only `.env.example` with placeholder values is committed. No secret appears in logs, error messages, or the repository.
8. **Honest communication.** Never present the system as a production security product. State known limits: lab-generated 2017 data, known dataset artefacts, shortcut risk from ports, uncalibrated scores, low-support classes.

19. Edge-Case Catalogue (Required Behaviour)

Area	Condition	Required behaviour
Data	Raw folder missing or empty	STOP with 0.4(a) message, exit code 2
Data	Mixed variant A and B files	Raise error naming offending files
Data	Column sets differ across files	Intersection if at least 90% of union, else STOP; record dropped columns
Data	Label absent from `label_map.yaml`	Raise `UnmappedLabelError` listing all unmapped labels
Data	File contains only benign rows	Allowed
Data	All timestamps unparsable	`timeline_mode = sequence`; no time filter; message in UI
Data	Timestamp outside expected range	Set to NaT, count in report
Data	IPv6 or malformed IP strings	IPv6 accepted; malformed IP in dataset becomes null with a count; never a crash

Area	Condition	Required behaviour
Data	Port outside 0 to 65535 or non-numeric	Set null in metadata; keep the row
Data	Memory exhausted	Process per file, float32, `--fast`; otherwise STOP with a message naming the stage
Split	Family has zero rows in train	`SplitError`
Split	Family has zero rows in val or test	Allowed; excluded from per-class metrics and flagged
Split	Very small class (fewer than `min_per_split`)	Allowed; flagged `low_support`; no precision or F1 headline claims about it
Training	Validation has no positive rows	STOP (cannot choose a threshold)
Training	Many identical scores (saturated forest scores)	Threshold search uses unique score values; `>=` rule applied consistently; documented in model card
Training	XGBoost multiclass labels not contiguous	Always encode with the saved `LabelEncoder` fitted on train families
Training	Train-versus-validation gap above 0.05 F1	Warning in model card and `reports/val_metrics.json`
Training	Reported metric would be a perfect 1.0	Add a model-card note listing the likely causes (duplicates, shortcut features, repeated patterns) and require the novelty diagnostic to be shown beside it
Inference	Missing feature	422 `missing_features`; never silently fill
Inference	Null or NaN value	Imputed and counted
Inference	Value outside training range	Passed through; counted in `out_of_range_feature_count`
Inference	Feature names with different spacing	Normalized before matching
Inference	Concurrent requests	Safe because the predictor is read-only; `n_jobs=1`

Area	Condition	Required behaviour
Inference	`ACTIVE` points to a missing or corrupt bundle	Degraded mode, 503 on predictions
Inference	Model changed on disk while API runs	Not hot-reloaded; restart required (state in README)
UI	Zero alerts in run	Show `No alerts predicted in this run`; hide empty charts
UI	Run where every flow is malicious	Charts and KPIs still render; no division by zero
UI	Very large run	Aggregate before plotting; paginate tables
UI	Rerun of the script by Streamlit while analysing	Duplicate-run guard (14.11)
UI	Selected run directory deleted	Clear message, reselect default
UI	Browser refresh	Selected run kept through the `run` query parameter (SHOULD)
Storage	Failure while writing a run	Write into a temp directory and rename atomically; delete the temp directory on error
Storage	Two sessions create runs at the same second	Random suffix in `run_id` prevents collisions
Storage	PostgreSQL unreachable	Fall back to files with a visible warning
Reproducibility	Library versions differ from bundle	Warning plus `version_mismatch` flag